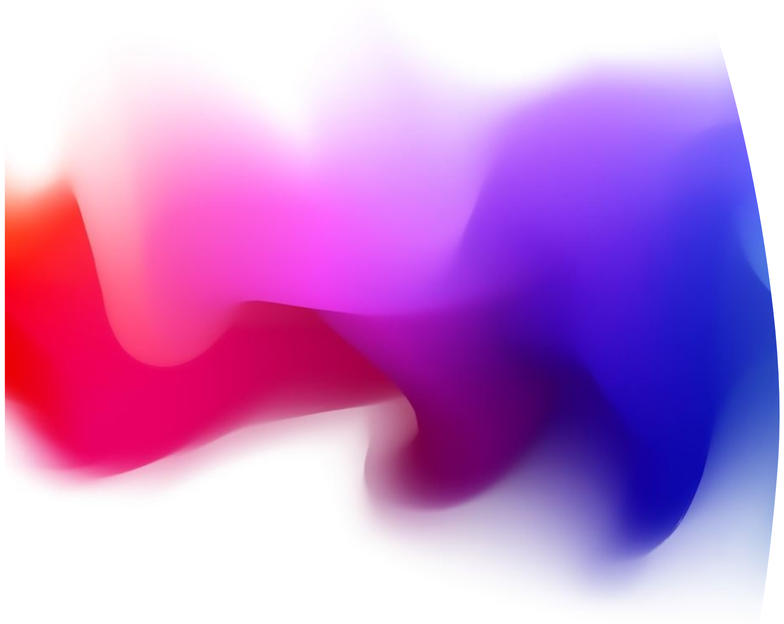


Lecture 14 -- Assignment Briefing and Adversarial Attack & Defence on Deep Learning Models

Prof Xiaowei Huang

https://cgi.csc.liv.ac.uk/~xiaowei/

(Attendance Code: **683143**)

Information

2 (assignments) in 1 (submission)

Two scores

Deadline:

17:00, 5th December, 2025

17:00, 12th December, 2025

This is the final deadline

Section D of Textbook: https://link.springer.com/book/10.1007/978-981-19-6814-3

Attendance Code: 683143

Overall Aims

This is a student competition to address two key issues in modern deep learning, i.e.,

O1 how to find better adversarial attacks, and

O2 how to train a deep learning model with better robustness to the adversarial attacks.

We provide a **template code** (Competition/Competition.py), where there are two code blocks corresponding to the training and the attack, respectively.

https://github.com/xiaoweih/AISafetyLectureNotes/tree/main/Competition

The two code blocks are filled with the simplest implementations representing the baseline methods, and the participators are expected to replace the baseline methods with their own implementations, to achieve better performance regarding the above O1 and O2.

File Structure

File Structure

```
Competition/
├── Competition.py           # Your submission file
├── Reference_attacks.py     # Baseline attacks (for testing only)
├── Evaluate_defence.py      # Test defence score
├── Evaluate_attack.py       # Test attack score
└── Defenders/               # Reference models (download from Canvas – see Setup)
    ├── 0.pt
    ├── 1.pt
    └── ... (10 models total)
```

Submission Guideline

- Must carefully follow the instructions in
  - D.3 Implementation Actions
  - D.1 Submissions

copied to the following slides

- Your submission will not be marked if you do not follow the instructions and/or the markers cannot run your code.

Attendance Code: 683143

Code Template Explanation

Attendance Code: 683143

D.2.1 Load Packages

First of all, the following code piece imports a few packages that are needed.

```python
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torch.optim as optim
import torchvision
from torchvision import transforms
from torch.autograd import Variable
import argparse
import time
import copy
```

Do not randomly import packages; the marking computer might not install your packages

Note: You can add necessary packages for your implementation.

Attendance Code: 683143

D.2.2 Define Competition ID

The below line of code defines the student number. By replacing it with your own student number, it will automatically output the file *i* **.pt** once you trained a model.

```
# input id
id_ = 1000
```

D.2.3 Set Training Parameters

```python
# setup training parameters
parser = argparse.ArgumentParser(description='PyTorch MNIST Training')
parser.add_argument('--batch-size', type=int, default=128, metavar='N',
                    help='input batch size for training (default: 128)')
parser.add_argument('--test-batch-size', type=int, default=128, metavar='N',
                    help='input batch size for testing (default: 128)')
parser.add_argument('--epochs', type=int, default=10, metavar='N',
                    help='number of epochs to train')
parser.add_argument('--lr', type=float, default=0.01, metavar='LR',
                    help='learning rate')
parser.add_argument('--no-cuda', action='store_true', default=False,
                    help='disables CUDA training')
parser.add_argument('--seed', type=int, default=1, metavar='S',
                    help='random seed (default: 1)')
args = parser.parse_args(args=[])
```

Attendance Code: 683143

D.2.5 Loading Dataset and Define Network Structure

```
#################################################don't change the below code #################################################

train_set = torchvision.datasets.FashionMNIST(root='data', train=True, download=True, transform=transforms.Compose([transforms.ToTensor()]))
train_loader = DataLoader(train_set, batch_size=args.batch_size, shuffle=True)

test_set = torchvision.datasets.FashionMNIST(root='data', train=False, download=True, transform=transforms.Compose([transforms.ToTensor()]))
test_loader = DataLoader(test_set, batch_size=args.batch_size, shuffle=True)

# define fully connected network
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(28*28, 128)
        self.fc2 = nn.Linear(128, 64)
        self.fc3 = nn.Linear(64, 32)
        self.fc4 = nn.Linear(32, 10)

    def forward(self, x):
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)
        x = F.relu(x)
        x = self.fc3(x)
        x = F.relu(x)
        x = self.fc4(x)
        output = F.log_softmax(x, dim=1)
        return output

#################################################end of "don't change the below code" #################################################
```

Attendance Code: 683143

D.2.6 Adversarial Attack

```python
'generate adversarial data, you can define your adversarial method'
def adv_attack(model, X, y, device):
    X_adv = Variable(X.data)

    ################################################Note: below is the place you need to edit to implement your own attack algorithm
    ################################################

    random_noise = torch.FloatTensor(*X_adv.shape).uniform_(-0.1, 0.1).to(device)
    X_adv = Variable(X_adv.data + random_noise)

    ################################################ end of attack method
    ################################################

    return X_adv
```

The part is the place needing your implementation, for O1. In the template code, it includes a baseline method which uses random sampling to find adversarial attacks. You can only replace the middle part of the function with your own implementation (as indicated in the code), and are not allowed to change others.

Attendance Code: 683143

D.2.8 Adversarial Training

```python
#train function, you can use adversarial training
def train(args, model, device, train_loader, optimizer, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        data = data.view(data.size(0),28*28)

        #use adverserial data to train the defense model
        #adv_data = adv_attack(model, data, target, device=device)

        #clear gradients
        optimizer.zero_grad()

        #compute loss
        #loss = F.nll_loss(model(adv_data), target)
        loss = F.nll_loss(model(data), target)

        #get gradients and update
        loss.backward()
        optimizer.step()

#main function, train the dataset and print train loss, test loss for each epoch
def train_model():
    model = Net().to(device)

    ################################################################
    ## Note: below is the place you need to edit to implement your own training algorithm
    ##       You can also edit the functions such as train(...).
    ################################################################

    optimizer = optim.SGD(model.parameters(), lr=args.lr)
    for epoch in range(1, args.epochs + 1):
        start_time = time.time()

        #training
        train(args, model, device, train_loader, optimizer, epoch)

        #get trnloss and testloss
        trnloss, trnacc = eval_test(model, device, train_loader)
        advloss, advacc = eval_adv_test(model, device, train_loader)

        #print trnloss and testloss
        print('Epoch '+str(epoch)+': '+str(int(time.time()-start_time))+'s', end=', ')
        print('trn_loss: {:.4f}, trn_acc: {:.2f}%'.format(trnloss, 100. * trnacc), end=', ')
        print('adv_loss: {:.4f}, adv_acc: {:.2f}%'.format(advloss, 100. * advacc))

    adv_tstloss, adv_tstacc = eval_adv_test(model, device, test_loader)
    print('Your estimated attack ability, by applying your attack method on your own trained model, is: {:.4f}'.format(1/adv_tstacc))
    print('Your estimated defence ability, by evaluating your own defence model over your attack, is: {:.4f}'.format(adv_tstacc))
    ################################################
    ## end of training method
    ################################################

    #save the model
    torch.save(model.state_dict(), str(id_)+'.pt')
    return model
```

Attendance Code: 683143

D.2.10 Supplementary Code for Test Purpose

In addition to the above code, we also provide two lines of code for testing purpose. You must comment them out in your submission. The first line is to call the **train_model()** method to train a new model, and the second is to check the quality of attack based on a model.

That is, comment out this line in your submission.

```
#Comment out the following command when you do not want to re-train the model
#In that case, it will load a pre-trained model you saved in train_model()
model = train_model()

#Call adv_attack() method on a pre-trained model
#The robustness of the model is evaluated against the infinite-norm distance measure
#!!! Important: MAKE SURE the infinite-norm distance (epsilon p) less than 0.11 !!!
p_distance(model, train_loader, device)
```

Attendance Code: 683143

D.1 Submissions

Every student with student number i will submit a package i**.zip**, which includes two files:

1. i**.pt**, which is the file to save the trained model, and
2. competition_i**.py**, which is your script after updating the two code blocks in **Competition.py** with your implementations.

NB: Please carefully follow the naming convention as indicated above, and we will not accept submissions which do not follow the naming convention.

D.3 Implementation Actions

Below, we summarise the actions that need to be taken for the completion of a submission:

Do not change

1. You must assign the variable **id_** with your student ID i;
2. You need to update the **adv_attack** function with your adversarial attack method;
3. You may change the hyper-parameters defined in **parser** if needed;
4. You must make sure the perturbation distance less than **0.11**, (which can be computed by **p_distance** function);
5. You need to update the **train_model** function (and some other functions that it called such as **train**) with your own training method;
6. You need to use the line "model = train_model()" to train a model and check whether there is a file i**.pt**, which stores the weights of your trained model;
7. You must submit i**.zip**, which includes two files i**.pt** (saved model) and competition_i**.py** (your script).

If the final filename is changed by the submission system, that's fine

Attendance Code: 683143

D.3.1 Sanity Check

Please make sure that the following constraints are satisfied. Your submission won't be marked if they are not followed.

- Submission file: please follow the naming convention as suggested above.
- Make sure your code can run smoothly.
- Comment out the two lines "model = train_model()" and "p_distance(model, train_loader , device)", which are for test purpose.

Your submission will not be marked if you do not follow the instructions and/or the markers cannot run your code.

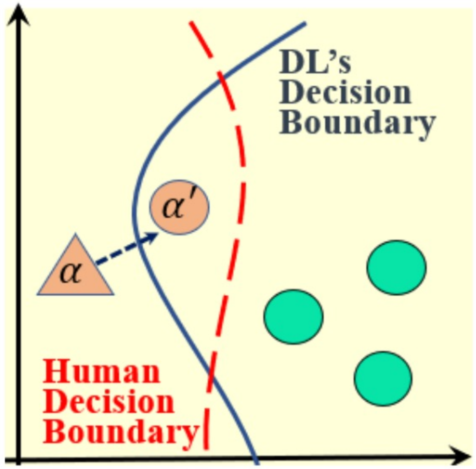
Attendance Code: 683143

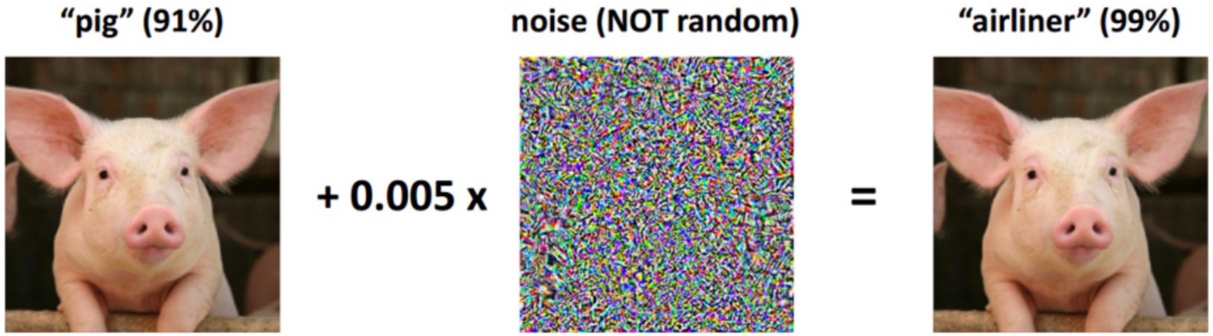
Theoretical Knowledge on Attack and Defence of Deep Learning

Attendance Code: 683143

What is Adversarial Example

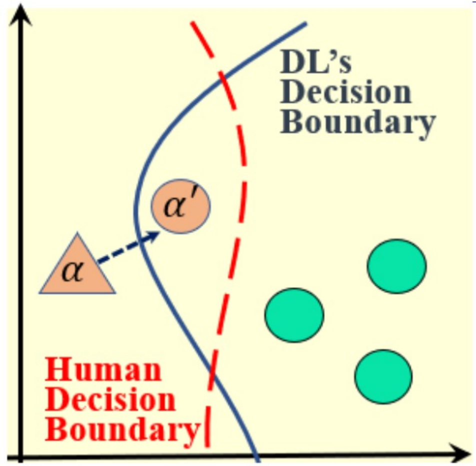
- Input: DL model f
  A correctly-classified, genuine example α
- Aim: find a perturbed example α', such that
  - f produces a different decision on α'
  - Human will produce a same decision on α and α'





Attendance Code: 683143

What is Adversarial Example



Problem: Given a DL model f and genuine example α, find α' such that

- $f(\alpha') \neq f(\alpha)$
- $D_{Human}(\alpha') = D_{Human}(\alpha)$

How to approximate human decision?

- Use certain distance metric to assure α' and α are small enough

How to search α' such that $f(\alpha') \neq f(\alpha)$?

- Design certain objective functions for minimization

What kind of information is required from DL model f?

- White-box v.s. Black-box

Attendance Code: 683143

Topics

Attacks

Defence

Attack Definition

One of earliest adversarial attack: optimization based formulation with L_2-norm metric

- Model $f : \mathbb{R}^{s_1} \to \{1 \dots s_K\}$ with s_K labels
- $x \in \mathbb{R}^{s_1} = [0,1]^{s_1}$ is an input
- $t \in \{1 \dots s_K\}$ is a target misclassification label

Find the adversarial perturbation r via

$$\begin{aligned} & \min \|r\|_2 && \text{assure human-decision unchanged} \\ s.t.\quad & \arg\max_l f_l(x+r) = t && \text{assure misclassification} \\ & x + r \in \mathbb{R}^{s_1} && \text{assure perturbed image feasible} \end{aligned} \tag{1}$$

- Solved by L-BFGS, Establish this direction

Attendance Code: 683143

Attack by Adding Noise (from assignment code template)

```
'generate adversarial data, you can define your adversarial
    method'
def adv_attack(model, X, y, device):
    X_adv = Variable(X.data)

    ############################################Note: below is
    the place you need to edit to implement your own attack
    algorithm
    ############################################

    random_noise = torch.FloatTensor(*X_adv.shape).uniform_(-0.1,
     0.1).to(device)
    X_adv = Variable(X_adv.data + random_noise)

    ############################################ end of attack
    method
    ############################################

    return X_adv
```

FGSM Attack

- Fast Gradient Sign Method can find adversarial perturbations with a fixed L∞-norm constraint very efficiently

- θ: the model parameters,
- x, y: the input and the label
- $J(\theta, x, y)$: the loss function

Find adversarial perturbation r by linearizing the loss function around the current value of θ,

$$r = \epsilon \ \mathrm{sign}\left(\nabla_x J(\theta, x, y)\right) \tag{2}$$

- A **one-step modification** to all pixel values to increase the loss function with a L_∞-norm constraint ϵ

Attendance Code: 683143

PGD Attack

Kurakin et al (2016). Adversarial machine learning at scale. arXiv:1611.01236

Idea: Iterative gradient ascent to generate strongest adversarial examples, followed by projection back to the feasible region

Updating rule:

$$x^{t+1} = \Pi_{S_x}(x^t + \alpha \cdot \mathrm{sgn}(\nabla_x L(x^t, y; \theta))),$$

where $\Pi_{S_x}(\cdot)$ projects the inputs onto the region S_x.

Training (from assignment code template)

```
#main function, train the dataset and print train loss, test loss
     for each epoch
def train_model():
    model = Net().to(device)

    #
    ################################################################
    ## Note: below is the place you need to edit to implement
    your own training algorithm
    ##       You can also edit the functions such as train(...).
    #
    ################################################################

    optimizer = optim.SGD(model.parameters(), lr=args.lr)
    for epoch in range(1, args.epochs + 1):
        start_time = time.time()

        #training
        train(args, model, device, train_loader, optimizer, epoch
    )

        #get trnloss and testloss
        trnloss, trnacc = eval_test(model, device, train_loader)
        advloss, advacc = eval_adv_test(model, device,
    train_loader)
```

```
#train function, you can use adversarial training
def train(args, model, device, train_loader, optimizer, epoch):
    model.train()
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        data = data.view(data.size(0),28*28)

        #use adverserial data to train the defense model
        #adv_data = adv_attack(model, data, target, device=device
    )

        #clear gradients
        optimizer.zero_grad()

        #compute loss
        #loss = F.nll_loss(model(adv_data), target)
        loss = F.nll_loss(model(data), target)

        #get gradients and update
        loss.backward()
        optimizer.step()
```

Adversarial Training via Robust Optimisation

Madry et al (ICLR 2018). Towards Deep Learning Models Resistant to Adversarial Attacks

Idea: solving a minimax optimisation problem through SGD training

$$\min_{\theta}\{\mathbb{E}_{(x,y)\sim\mathcal{D}}[\max_{x'\in S_x} L(x',y;\theta)]\},$$

- (x, y) - clean training data samples $x \in \mathbb{R}^n$ with labels $y \in [k]$ drawn from the dataset $\mathcal{D}$
- $L(\cdot)$ - loss function with model parameter $\theta \in \mathbb{R}^m$
- $x' \in \mathbb{R}^n$ - perturbed samples in a feasible region $S_x \triangleq \{z : z \in B(x,\epsilon) \cap [-1.0, 1.0]^n\}$
- e.g., $B(z,\epsilon) \triangleq \{z : \|x - z\|_p \leq \epsilon\}$ - the ℓ_p-ball at center x with radius ϵ.

Adversarial Attacks as Inner Maximisation

- Outer minimisation can be simulated by SGD training

$$\min_{\theta}\{\frac{1}{N}\sum_{i=1}^{N}[\max_{x'\in S_x} L(x',y;\theta)]\},$$

- How to compute gradient of a maximisation?
  - Danskin's Theorem

$$\nabla_\theta \max L(x',y;\theta) = \nabla_\theta L(x^*,y;\theta)$$

    where $x^* = \arg\max L(x',y;\theta)$
- Inner maximisation $\max_{x'\in S_x} L(x',y;\theta)$ can be simulated by finding the worst-case adversarial attacks:
  - Fast Gradient Method (FGM)
  - Projected Gradient Method (PGM):

Guidance on debugging

- AISafetyLectureNotes/Competition/README_baseline.md

```
(aisafety) xiaowei@Xiaoweis-MacBook-Pro-3 AISafetyLectureNotes % git pull
Updating 8b71eab..470b0d2
Fast-forward
 Competition/.gitignore                         |  25 +++++
 Competition/Competition.ipynb                  | 275 ++++++++++++++++++++++++++++++++---------------
 Competition/Competition.py                     |  29 +++--
 Competition/Evaluate_attack.py                 | 189 ++++++++++++++++++++++++++++++++
 Competition/Evaluate_defence.py                | 164 ++++++++++++++++++++++++++++
 Competition/README_baseline.md                 | 193 +++++++++++++++++++++++++++++++++
 Competition/Reference_attacks.py               |  65 ++++++++++++
 Competition/{ => archive}/Evaluation_matrix.py |   0
 Competition/{ => archive}/Mark.py              |   0
 Competition/archive/README.md                  |  16 +++
 10 files changed, 862 insertions(+), 94 deletions(-)
 create mode 100644 Competition/.gitignore
 create mode 100644 Competition/Evaluate_attack.py
 create mode 100644 Competition/Evaluate_defence.py
 create mode 100644 Competition/README_baseline.md
 create mode 100644 Competition/Reference_attacks.py
 rename Competition/{ => archive}/Evaluation_matrix.py (100%)
 rename Competition/{ => archive}/Mark.py (100%)
 create mode 100644 Competition/archive/README.md
```

```
Competition/
├── Competition.py          # Your submission file
├── Reference_attacks.py    # Baseline attacks (for testing only)
├── Evaluate_defence.py     # Test defence score
├── Evaluate_attack.py      # Test attack score
└── Defenders/              # Reference models (download from Canvas - see Setup)
    ├── 0.pt
    ├── 1.pt
    └── ... (10 models total)
```

Note: The `Defenders/` folder is **not included in GitHub**. You must download `Defenders.zip` from Canvas → COMP219 Module → Lecture 14, then extract it to the Competition directory.

```
(aisafety) xiaowei@Xiaoweis-MacBook-Pro-3 AISafetyLectureNotes % python Competition/Competition.py
/opt/anaconda3/envs/aisafety/lib/python3.13/site-packages/torch/nn/_reduction.py:51: UserWarning: size_average and reduce ar
gs will be deprecated, please use reduction='sum' instead.
  warnings.warn(warning.format(ret))
Epoch 1: 4s, trn_loss: 1.9006, trn_acc: 38.37%, adv_loss: 1.9011, adv_acc: 38.38%
Epoch 2: 4s, trn_loss: 1.0741, trn_acc: 61.12%, adv_loss: 1.0752, adv_acc: 61.15%
Epoch 3: 4s, trn_loss: 0.8682, trn_acc: 68.08%, adv_loss: 0.8692, adv_acc: 68.06%
Epoch 4: 4s, trn_loss: 0.7805, trn_acc: 69.71%, adv_loss: 0.7822, adv_acc: 69.68%
Epoch 5: 5s, trn_loss: 0.7016, trn_acc: 74.81%, adv_loss: 0.7034, adv_acc: 74.76%
Epoch 6: 4s, trn_loss: 0.6594, trn_acc: 76.19%, adv_loss: 0.6614, adv_acc: 76.08%
Epoch 7: 4s, trn_loss: 0.6244, trn_acc: 77.26%, adv_loss: 0.6272, adv_acc: 77.13%
Epoch 8: 4s, trn_loss: 0.5785, trn_acc: 79.36%, adv_loss: 0.5820, adv_acc: 79.27%
Epoch 9: 4s, trn_loss: 0.5520, trn_acc: 80.75%, adv_loss: 0.5555, adv_acc: 80.62%
Epoch 10: 4s, trn_loss: 0.5375, trn_acc: 81.05%, adv_loss: 0.5409, adv_acc: 80.88%

================================================================================
Training completed! Model saved as: 1000.pt
================================================================================

To evaluate your attack and defence scores:
  1. Test your attack:  python Evaluate_attack.py
  2. Test your defence: python Evaluate_defence.py

Make sure to:
  - Update attack method in Evaluate_attack.py
  - Update model file path in Evaluate_defence.py
  - Verify epsilon constraint < 0.11 (see p_distance output below)
================================================================================

epsilon p:  tensor(0.1000)
```

Attendance Code: 683143

```
(aisafety) xiaowei@Xiaoweis-MacBook-Pro-3 Competition % python Evaluate_attack.py
/Users/xiaowei/Library/CloudStorage/OneDrive-TheUniversityofLiverpool/code/AISafetyLectureNotes/Competition/Evaluate_attack.py:82: SyntaxWarning: invalid escap
e sequence '\D'
  defenders_path = ".\Defenders"  # change this to your extracted Defenders folder path
================================================================================
COMP219 Assignment - Attack Evaluation
================================================================================
Defenders path: .\Defenders
Device: cpu
================================================================================

ERROR: Defenders folder not found at '.\Defenders'

Please:
1. Go to Canvas → COMP219 Module → Lecture 14
2. Download 'Defenders.zip'
3. Extract the zip file
4. Update 'defenders_path' variable in this script
(aisafety) xiaowei@Xiaoweis-MacBook-Pro-3 Competition % vi Evaluate_attack.py
```

Attendance Code: 683143

```
(aisafety) xiaowei@Xiaoweis-MacBook-Pro-3 Competition % python Evaluate_attack.py
================================================================================
COMP219 Assignment - Attack Evaluation
================================================================================
Defenders path: ./Defenders
Device: cpu
================================================================================
Found 10 reference models
================================================================================

Evaluating your attack...
  [1/10] 0.pt... 86.95%
  [2/10] 1.pt... 83.96%
  [3/10] 2.pt... 78.53%
  [4/10] 3.pt... 83.79%
  [5/10] 4.pt... 78.24%
  [6/10] 5.pt... 80.12%
  [7/10] 6.pt... 81.60%
  [8/10] 7.pt... 80.88%
  [9/10] 8.pt... 60.70%
  [10/10] 9.pt... 82.13%

================================================================================
ATTACK MATRIX (Your Attack vs Reference Models)
================================================================================

Model                Robust Accuracy      Attack Score (1/acc)
--------------------------------------------------------------------------------
0.pt                  86.95%                1.15
1.pt                  83.96%                1.19
2.pt                  78.53%                1.27
3.pt                  83.79%                1.19
4.pt                  78.24%                1.28
5.pt                  80.12%                1.25
6.pt                  81.60%                1.23
7.pt                  80.88%                1.24
8.pt                  60.70%                1.65
9.pt                  82.13%                1.22
--------------------------------------------------------------------------------
Mean                  79.69%                1.27
================================================================================

================================================================================
ATTACK SCORE
================================================================================
Your Attack Score: 1.2661
================================================================================

Note:
  - Higher score is better (stronger attack)
  - Attack Score = mean(1/robust_accuracy)
  - Lower robust accuracy → stronger attack → higher score
  - Final grading will normalize scores across ALL students
```

```
(aisafety) xiaowei@Xiaoweis-MacBook-Pro-3 Competition % python Evaluate_defence.py
================================================================================
COMP219 Assignment - Defence Evaluation
================================================================================
Model: 1000.pt
Device: cpu
Testing against 3 attack(s)
================================================================================

Evaluating...
  [1/3] FGSM... 9.27%
  [2/3] PGD-5... 7.01%
  [3/3] PGD-20... 4.49%

================================================================================
DEFENCE MATRIX (Your Model vs Reference Attacks)
================================================================================

Attack          Robust Accuracy
--------------------------------------------------------------------------------
FGSM              9.27%
PGD-5             7.01%
PGD-20            4.49%
--------------------------------------------------------------------------------
Mean              6.92%
================================================================================

================================================================================
DEFENCE SCORE
================================================================================
Your Defence Score: 0.0692
================================================================================

Note:
  - Higher score is better (more robust model)
  - Final grading will normalize scores across ALL students
  - This self-test uses reference attacks only
```